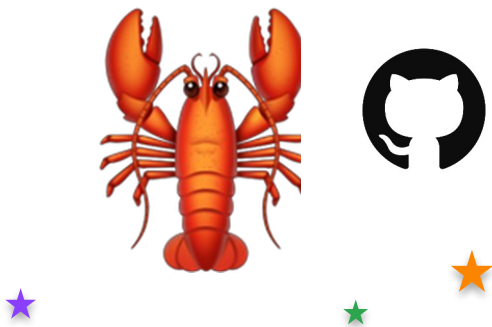




开源项目潜力发现系统

从 **GitHub** 早期信号中挖掘值得观察的开源项目



目标更早一步 —— 在仓库创建后的早期窗口，识别值得关注的潜力项目

↘ GitHub Trending

看见的是：已经获得关注的项目
问题是：可能错过早期机会

启发

✧ OpenClaw Radar

观察的是：前 30 天的项目行为
输出的是：可追踪的候选清单

研究支撑：相关研究表明这条路是可行的

Predicting the Popularity of GitHub Repositories

PROMISE'16

用星标时间序列 + 多元回归即可
预测仓库流行度

→ 流行度是可建模的量

Understanding the Factors that Impact the Popularity

ICSME'16

2,279 仓库 4 类增长模式；
语言、领域显著影响星标

→ 项目特征带预测信号

What Makes a Popular Academic AI Repository?

EMSE / ICSE'21

1,149 学术 AI 仓库；21 特征中 11 个
在热门 vs 冷门间显著差异

→ 早期工程实践可识别

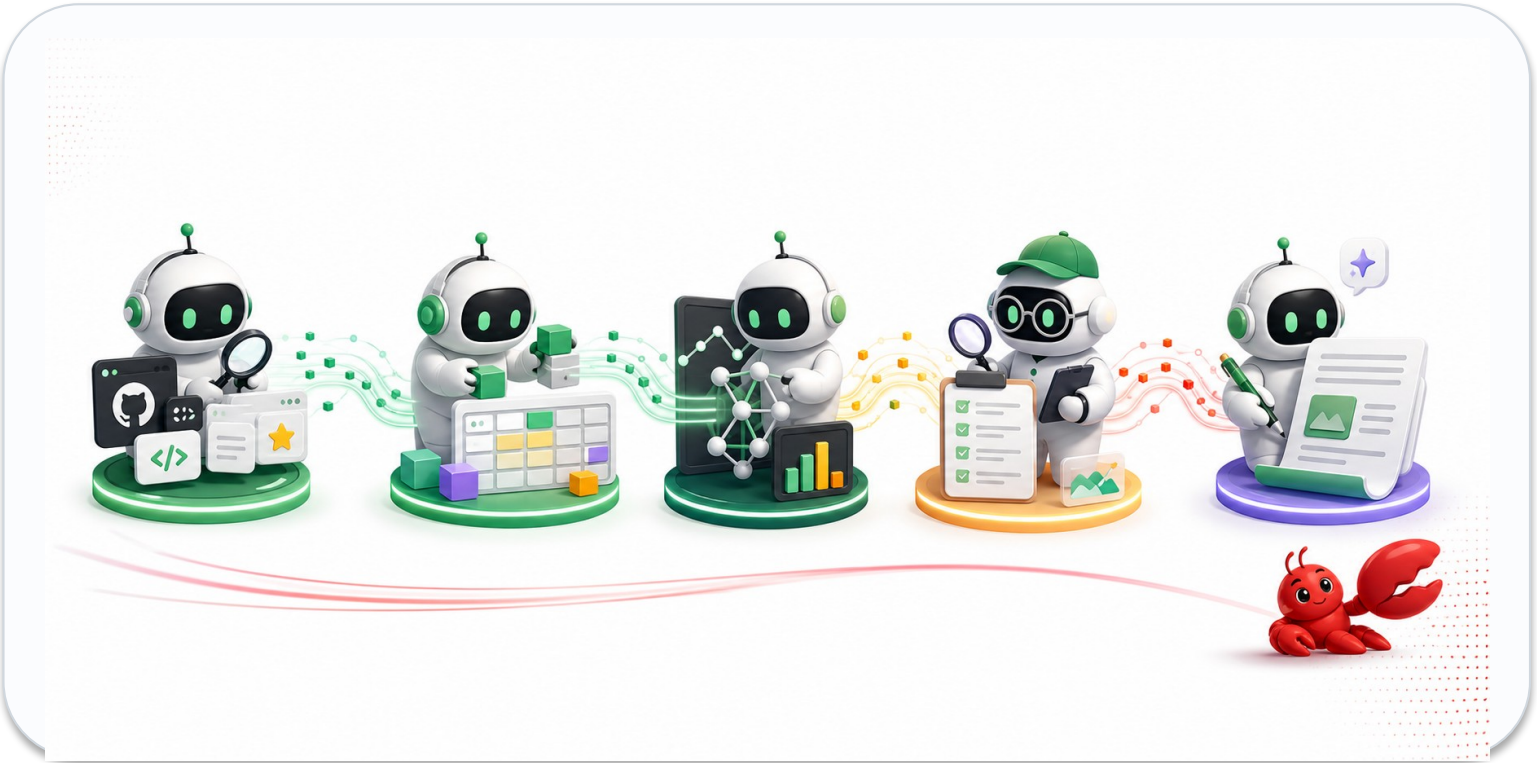
不是把脚本串起来，而是由编排器调度多个角色，每个角色留下可检查的证据文件。

OpenClaw Orchestrator

负责调度，不直接写结论

- 1 检查状态 missing / stale
- 2 选择技能 skill routing
- 3 运行角色 subprocess events
- 4 刷新控制台 dashboard visible

核心价值：可复现流程 + 可追踪产物 + 受约束推理



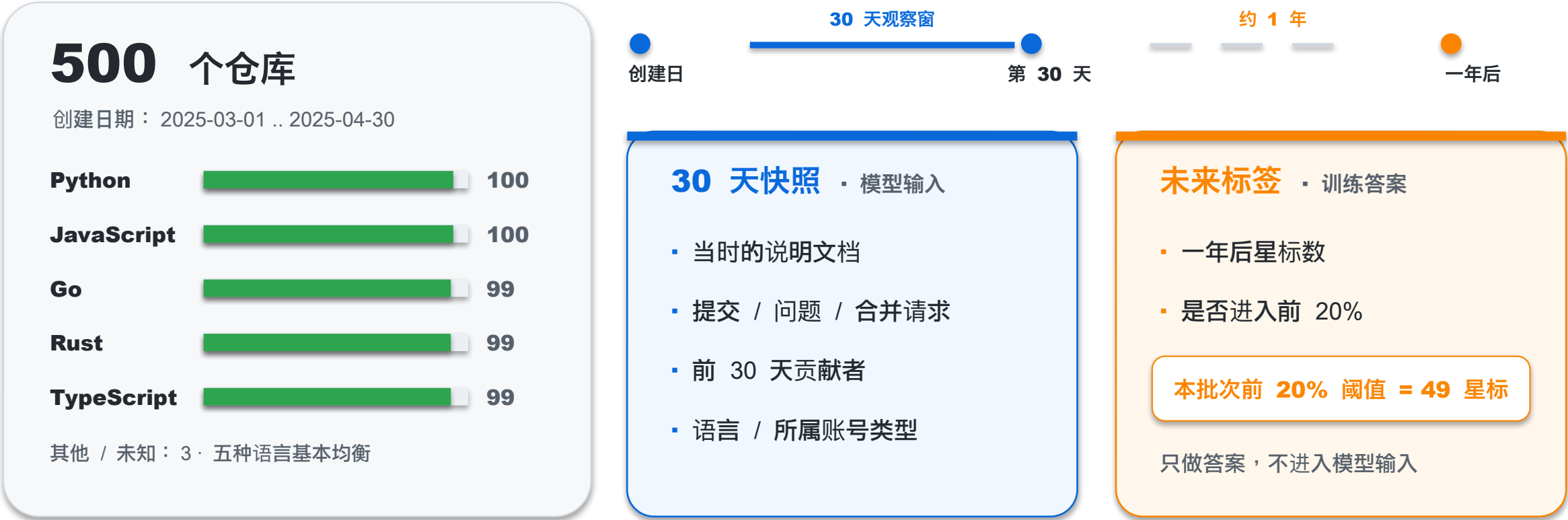
各角色处理不同任务，最终把原始数据 → 特征表 → 模型结果 → 事实摘要 → 洞察报告串成完整证据链。

每个 **Agent** 都不是一句 **prompt**，而是一套明确的输入、脚本、输出和事件记录。



OpenClaw 的价值：把“能跑一次的脚本”升级成“可检查、可追踪、可复现的多角色流程”。

每个仓库拆成两部分：创建后 30 天内能看到的信息，以及一年后的热度标签。



数据样例： 30 天快照是输入，未来标签是答案

用一条仓库记录说明数据切分：模型只能看左边，右边只用来判断是否成功。



模型不直接吃原始 **JSON**，而是把 **30** 天内的行为整理成五类可比较信号。



$6 + 1 + 4 + 4 + 4$

把不同仓库变成同一张表，
模型才能比较和排序。

6	语言信号	Python 、 JavaScript 、 Go 、 Rust 、 TypeScript 、 其他
1	归属信号	仓库属于组织账号，还是个人账号
4	活跃信号	提交、问题、合并请求、贡献者
4	文档信号	是否有说明文档、长度、图片、演示链接
4	派生信号	总活跃度、平均提交、请求 / 问题、有无请求

关键点：所有模型输入都来自项目创建后的早期窗口；一年后的星标数只用来生成训练答案。

模型训练：三类模型对比，逻辑回归整体最好，随机森林适合排序

训练阶段不只是跑分，还要保存模型文件，让 **Today Radar** 可以复用。

01

输入特征矩阵

500 个仓库 × 19 个特征

02

训练三类模型

逻辑回归 · 随机森林 · XGBoost

03

评估排序能力

5 折交叉验证 · AUC · 前 10 命中率

04

保存复用产物

模型文件 · 特征顺序 · 动态阈值

三个模型各自扮演什么角色？

LR

逻辑回归：线性、稳定

AUC 0.883

RF

随机森林：非线性、排序

PR-AUC
0.662

XGB

梯度提升：对照模型

P@10 0.80

模型比较不是选一个“唯一正确”，而是看排序、正例识别和应用需求。

指标解释：为什么不能只看一个 AUC

这个任务本质是候选发现，所以要同时看整体排序、正例识别、前排命中和时间泛化。

AUC

整体排序能力

随机取一个热门和冷门仓库，模型把热门排前面的概率。

0.883

PR-AUC

少数正例识别

正例只有约 20% 时，比准确率更能反映找潜力项目的能力。

0.662

P@10

前排候选质量

只看模型排在最前面的 10 个项目，贴近日常使用。

0.80

时间切分

时间泛化

按创建时间切分训练 / 测试，检验随机验证是否过于乐观。

+0.0004

展示时的解释顺序：**AUC** 证明可排序，**PR-AUC/P@10** 证明候选清单有意义，时间切分证明时间窗口稳定。

实验结果：早期信号确实能排序潜力仓库

模型最有价值的不是“预测每一个项目”，而是把候选项目排到更值得看的顺序。



结论：早期说明文档能持续提升排序；派生比率反而带来少量噪声。

单次问大模型也能写分析; **OpenClaw** 解决的是多步骤任务如何稳定运行、留下证据、可复现。

直接问大模型

- ✗ 把结果文件丢给模型总结
- ✗ 可能引用不存在的数字
- ✗ 每次回答不一定一致
- ✗ 难以接入完整流程

OpenClaw workflow

- ✓ 技能固定输入 / 输出契约
- ✓ 先算事实摘要, 再做解释
- ✓ 引用数字必须能在事实里校验
- ✓ 控制台展示状态与产物

先算事实,
再让模型解释

结果文件

事实诊断 → 结构化事实

洞察分析 → 受约束解释

OpenClaw 把模型训练和大模型分析, 组织成可审计、可复现的多智能体流程。

应用层不重新发明模型，而是复用同一套 19 个特征、训练好的模型文件和动态阈值。

01

近期仓库

创建 30-45 天

02

特征对齐

同一套 19 个特征

03

模型打分

随机森林输出关注分

04

行动分层

按训练集分位数划分

Top 5 候选榜单 (小规模实时扫描)

#1	lunashia/o-m_beatmap_trainer	0.935	Python	关注
#2	Kavishp7499/qp	0.718	Go	关注
#3	NotHarshhaa/mlflow-k8s-operator	0.718	Go	关注
#4	esengine/DeepSeek-Reasonix	0.718	Go	关注
#5	eSlider/bonsai-ollama	0.718	Go	关注

输出今日观察清单，后续可回看 star 增长

数据挖掘模型 + OpenClaw workflow + 今日候选应用

模型层

早期信号可以排序潜力仓库

AUC 0.883 · PR-AUC 0.662

系统层

OpenClaw 把复杂任务拆成
可复现步骤

技能 · 产物 · 控制台

应用层

Today Radar 复用同一套特
征和阈值

19 个特征 · 动态阈值

展示主线：先证明模型有效，再说明 **OpenClaw** 如何让这件事从一次实验变成可运行的系统。